Phantom type
Wiki: https://wiki.haskell.org/Phantom_type

Phantom type
A type with a **dummy type variable**.
e.g. MyType a = MyType Int a=dummy
Used like a **tag** i.e. a dumb label on the type

usage: tag with phantom type
e.g. Data.Fixed
Ref: https://hackage.haskell.org/package/base/docs/Data-Fixed.html
Represents a **fixed point** number.
i.e. real number represented by integer + decimal place

```
data Fixed a = MkFixed Int                -- Phantom
type a representing decimal place

class HasResolution a where
  resolution :: p a -> Integer

data E0
type Uni = Fixed E0
data E1
type Deci = Fixed Eq
...
```

usage

```
import Data.Fixed

v0 = MkFixed 12345 :: Uni       -- represents 12345
i.e. decimal at pos 0
resolution v0                   -- 1

v1 = MkFixed 12345 :: Deci      -- represents 1234.5
 i.e. point at pos 1
resolution v1                   -- 10

v2 = MkFixed 12345 :: Centi     -- represents 123.45
 i.e. point at pos 2
resolution v2                   -- 100
```

Proxy type
Ref: https://hackage.haskell.org/package/base/docs/Data-Proxy.html

```
-- Data.Proxy
data Proxy a = Proxy
```

Proxy
A **dummy value** to represent a **type**.
Used when a required type is originally NOT in the **signature**.
i.e. use Proxy as a **placeholder** for the type.

e.g. in type class methods

```
import Data.Proxy
class MyLabel a where
  -- myLabel :: String                    -- ERROR.
Signature MUST contain a
  myLabel :: Proxy a -> String           -- OK with
Proxy

instance MyLabel Int where myLabel _ = "This is an Int"
instance MyLabel Bool where myLabel _ = "This is a Bool"

myLabel (Proxy @Int)            -- This is an Int
myLabel (Proxy @Bool)           -- This is a Bool
```